

Week 4: Advanced Threat Detection & Web Security Enhancements

Project: OWASP Juice Shop - Security Enhancement

Week: 4

Name: Syeda Arooj Fatima

ID: DHC-1161

Internship Date: 14 June 2026

Overview

In Week 4, I focused on implementing advanced threat detection, hardening API security, and enforcing strict security headers on the OWASP Juice Shop application, building on the security utilities, password hashing, and JWT authentication implemented in Weeks 2 and 3.

1. Intrusion Detection & Monitoring

Implemented application-level brute-force detection in `utils/security.ts`, providing real-time monitoring and automatic blocking of repeated failed login attempts.

Functions Implemented

- `checkBruteForce(ip)` – checks whether an IP is currently locked out
- `recordFailedAttempt(ip)` – increments the failure counter and logs a warning once the threshold is reached
- `resetBruteForce(ip)` – clears the failure history for an IP after a successful login

Policy

- Maximum failed attempts: 5
- Lockout window: 15 minutes
- Response on lockout: HTTP 429 'Too many login attempts. Please try again later.'

Integration

`routes/secureLogin.ts` was updated so that every login request checks the brute-force status before processing credentials, records failures, and resets the counter on success.

CheckBruteForce:

```
;
// Check if the IP has too many failed login attempts
if (checkBruteForce(ip)) {
  return (res as any).status(429).json({ message: 'Too many login attempts. Please'
}
```

recordFailedAttempt:

```
const isValid = await comparePassword(password, user.password);
if (!isValid) {
  recordFailedAttempt(ip);
  return (res as any).status(401).json({ message: 'Invalid credentials' });
}
```

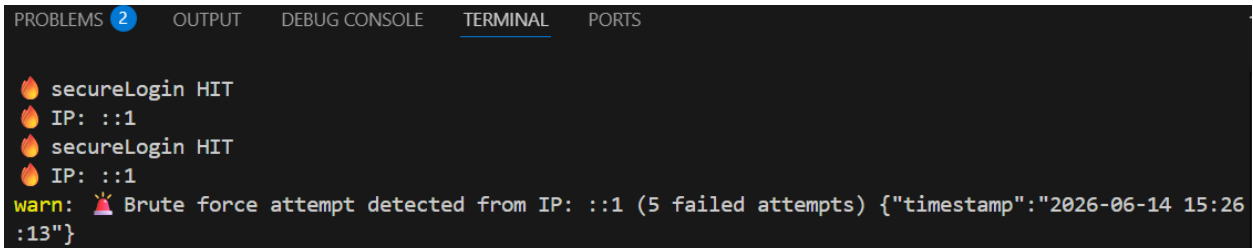
ResetBruteForce():

```
// Success - clear failed attempt history for this IP
resetBruteForce(ip);

const token = generateToken(user.id.toString());
```

When the threshold is reached, Winston logs the following warning:

🚨 Brute force attempt detected from IP: <ip> (5 failed attempts)



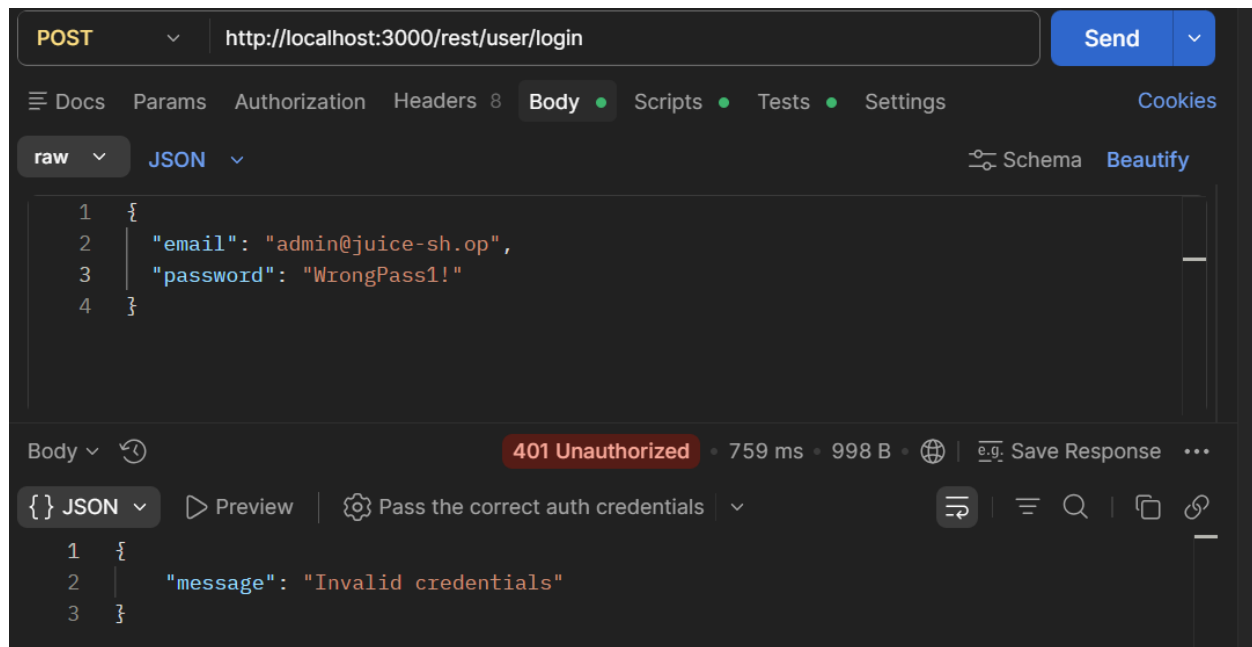
```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
🔥 secureLogin HIT
🔥 IP: ::1
🔥 secureLogin HIT
🔥 IP: ::1
warn: 🚨 Brute force attempt detected from IP: ::1 (5 failed attempts) {"timestamp": "2026-06-14 15:26:13"}
```

Testing Performed

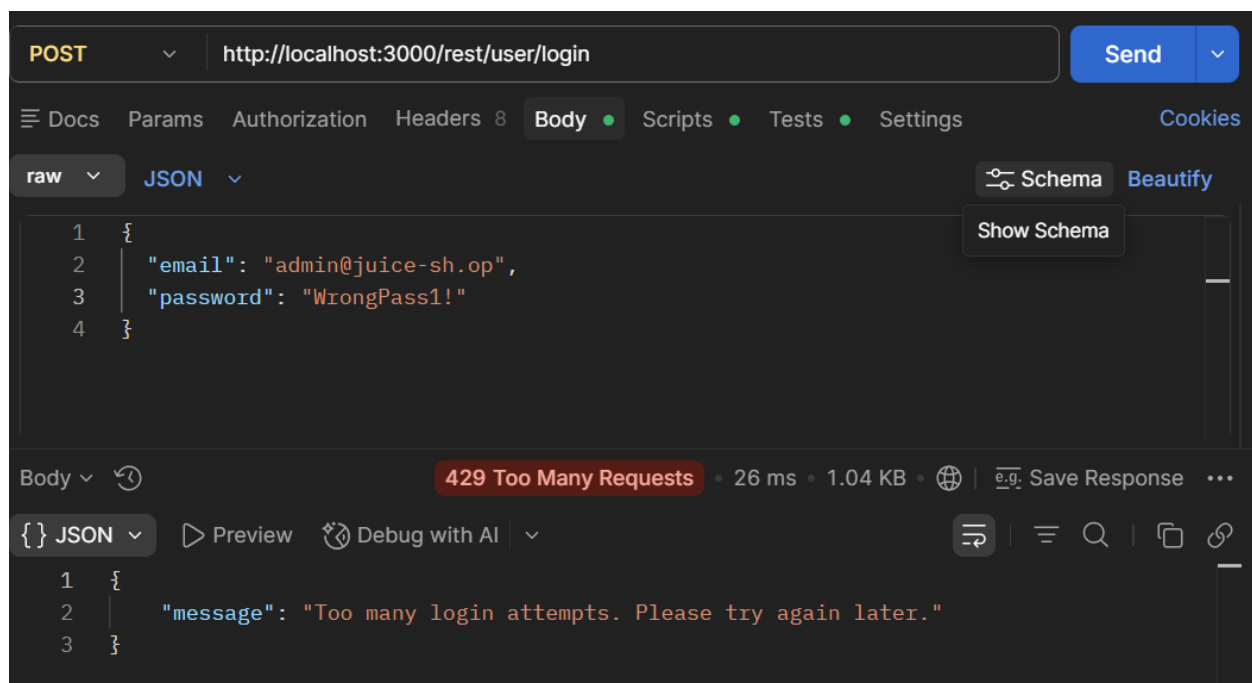
- Sent 5 consecutive login requests with valid-format but incorrect credentials → each returned 401 Unauthorized
- 6th request within the 15-minute window → returned 429 Too Many Requests
- Verified the brute-force warning was written to the Winston log after the 5th failed attempt
- Confirmed a correct login resets the failure counter for that IP

Proof:

On first attempt:



On many attempts:



2. API Security Hardening

Rate Limiting

Rate limiting via `express-rate-limit` (configured in Week 2/3) remains active and was verified to be applying correctly, visible in the `RateLimit-Policy` response header (`100;w=900`).

CORS Configuration

The original configuration allowed all origins:

```
app.options('*', cors(corsOptions));  
app.use(cors(corsOptions));
```

This was replaced with a restricted policy that only permits the application's own origin:

```
const corsOptions = {  
  origin: process.env.FRONTEND_URL || 'http://localhost:3000',  
  methods: ['GET', 'POST', 'PUT', 'DELETE'],  
  allowedHeaders: ['Content-Type', 'Authorization', 'x-api-key'],  
  credentials: true  
};  
  
app.options('*', cors(corsOptions));  
app.use(cors(corsOptions));
```

Verified via Postman — the response now includes:

Access-Control-Allow-Origin: <http://localhost:3000>

Access-Control-Allow-Credentials: true

Headers  200 OK • 118 ms • 5.12 KB •  e.g. Save Response ...	
Access-Control-Allow-Origin	http://localhost:3000
Vary	Origin, Accept-Encoding
Access-Control-Allow-Credentials	true

Authentication

Existing JWT-based authentication (Week 2) continues to protect sensitive routes via the verifyToken middleware. API key / OAuth-based protection for additional endpoints (e.g. /api/security/profile) is identified as a follow-up enhancement.

3. Security Headers & CSP Implementation

Helmet was reconfigured with explicit Content Security Policy directives and HSTS settings instead of relying on default values:

```

juice-shop > TS server.ts > hsts
58
59  // WEEK 4: HSTS + CSP
60
61  app.use(helmet({
62    contentSecurityPolicy: {
63      directives: {
64        defaultSrc: ["'self'"],
65        scriptSrc: ["'self'"],
66        styleSrc: ["'self'", "'unsafe-inline'"],
67        imgSrc: ["'self'", 'data:'],
68        objectSrc: ["'none'"],
69        upgradeInsecureRequests: []
70      }
71    },
72    hsts: {
73      maxAge: 31536000,
74      includeSubDomains: true,
75      preload: true
76    }
77  }));
78

```

Verified via Postman response headers:

Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; ...
 Strict-Transport-Security: max-age=31536000; includeSubDomains; preload

Key Files Created / Modified

File	Changes Made
utils/security.ts	Added checkBruteForce, recordFailedAttempt, resetBruteForce
routes/secureLogin.ts	Integrated brute-force checks, failure recording, and reset into login flow
server.ts	Restricted CORS to FRONTEND_URL; custom Helmet CSP and HSTS configuration
.env	Added FRONTEND_URL (alongside existing JWT_SECRET)

Challenges Faced & Solutions

- Challenge: A tsconfig.json incompatibility (ignoreDeprecations value) prevented a successful TypeScript build. Solution: Updated the value to match the installed TypeScript version.

- Challenge: After a successful rebuild, the server crashed on startup with a libuv assertion error on Windows (Node 24), traced to a built-in port-availability precondition check that detected the server's own listening socket as 'in use'. Solution: Removed the redundant checkIfPortIsAvailable check from validatePreconditions.ts.
- Challenge: After resolving the above, the server exited immediately with ERR_SERVER_ALREADY_LISTEN. Solution: Identified and removed a duplicate server.listen() call in server.ts left over from earlier edits.
- Challenge: Ensuring the brute-force counter only triggers on genuinely failed login attempts, not on input validation errors. Solution: Used a valid-format but incorrect password during testing so requests reach the credential comparison stage.

Conclusion

In Week 4, I successfully implemented advanced threat detection and further hardened the OWASP Juice Shop application:

- Real-time brute-force detection with automatic IP lockout and Winston logging
- Restricted CORS policy limiting requests to the application's own origin
- Custom Content Security Policy and HTTP Strict Transport Security headers
- Verified rate limiting remains active on all API endpoints

The OWASP Juice Shop application is now significantly more resilient against brute-force login attacks, cross-origin attacks, and script-injection-based attacks, building on the input validation, password hashing, and authentication hardening completed in previous weeks.

Future Improvements

- Move brute-force tracking from an in-memory Map to a shared store (e.g. Redis) with TTL expiry for multi-instance deployments
- Implement API key or OAuth-based authentication for additional protected endpoints
- Configure HTTPS so HSTS headers take full effect (carried over from Week 3)
- Automated security testing as part of a CI pipeline